INF1002
Programming Fundamentals
Lecture 3: Advanced Data Structure

Zhang Zhengchen
zhengchen.zhang@singaporetech.edu.sg

Task Priority and Assignments

Priority	Feature/Functions	Explanation	Estimated Time to Complete	Assigned To
P0	User Registration and Login	This is the core functionality that allows users to access the platform. It is essential for launch.	1 week	C
P0	Course Purchase	The platform's revenue depends on this feature. It's a critical function that needs early implementation.	2 weeks	B
P1	Recommendation System	This feature improves user experience by recommending courses, but it's not required for the initial release.	3 weeks	A
P1	User Reviews and Ratings	While helpful for other users when selecting courses, this feature can be added in later iterations.	1.5 weeks	C
P0	Document and Reflection		1-2 weeks	

- **Online study system**
- **P0 (High priority):** These are essential features for the project's core functionality and must be developed first for the project to launch successfully.
- **P1 (Low priority):** These are features that enhance the platform but are not necessary for the initial release. They can be developed in future iterations.

Review

- if elif elif else
  - Boolean expression
  - Conditional expression
    - *value_if_true* if condition else value_if_false
- while loop
  - Condition met or not
- for loop
  - Loop over items of a sequence
- for… else… and while… else…
- f-string
  - Alignment: > < ^
  - Width of a string
  - Precision of a float number

Outline

- Advanced Data Types
  - String
  - List
  - Tuple
  - Dictionary

Type	Samples
int	8,12,1024
float	2.3, 3.1415926
bool	True, False
str	'Hello, World! ', '3.1415926'
None	None
List	
Tuple	
Set	
Dictionary	
byte	

string

- An immutable sequence of Unicode characters.
  - Immutable
  - Sequence
  - Unicode
- Immutable vs mutable
  - Mutable: The value of the object can be changed, such as lists, dictionaries, and sets.
  - Immutable: The value of the object cannot be changed once it is created, such as strings, tuples, and integers.

string concatenation and repetition

- Strings are identified by single or double quotation marks:
  - 'Mike'
  - "Something interesting"
  - '73'
- The operator + is used for string concatenation:
  - 'Taylor' + 'Swift' evaluates to 'TaylorSwift'
  - 'Taylor' + ' '+ 'Swift' evaluates to 'Taylor Swift'
  - In your lab assignment, what will happen if you calculate *a* + *b* + *c* first, before converting *a,b,c* into float numbers with *float()*?
- Operator *

```
print('='*40)
print('#Some unused code here')
print('='*40)
```

```
========================================
#Some unused code here
========================================
```

string - a sequence of characters

- Sequence
- Membership Operator
  - in
  - not in

```python
name = 'john'
char = 't'
res = char in name
print(f'{char} in {name} = {res}')
char = 't'
res = char not in name
print(f'{char} not in {name} = {res}')
char = 't'
res = not char in name
print(f'not {char} in {name} = {res}')
char = 'h'
res = char in name
print(f'{char} in {name} = {res}')
```

```
t in john = False
t not in john = True
not t in john = True
h in john = True
```

This style is not recommended by PEP 8

string slicing

- Index of characters in a string
  - First character is indexed with 0.
  - Last character is indexed with len(s)-1
  - A character is a string with length 1
  - s[index]
  - s[-1]
  - s[start : end]
    - end: exclusive

```python
print('01234')
s = 'abcde'
print(s)
print(s[0])
print(s[4])
print(s[-1])
print(s[0:3])
print(s[:3])
print(s[1:])
print(s[100])
```

```
01234
abcde
a
e
e
abc
abc
bcde
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
<ipython-input-19-b09acf130c69> in <cell line: 10>()
      8 print(s[:3])
      9 print(s[1:])
---> 10 print(s[100])

IndexError: string index out of range
```

Next steps: Explain error

string slicing

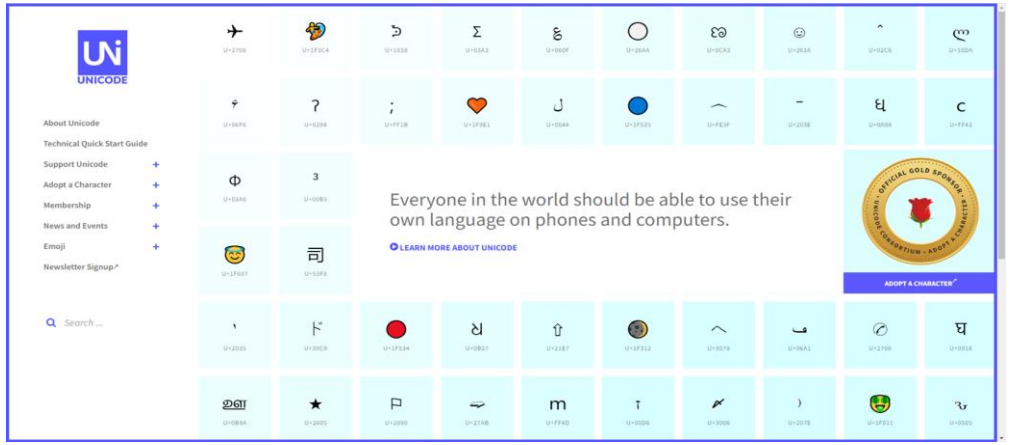
- Index of characters in a string
  - s[start : end : step]

```python
s='abcde'
sub = s[0:5:2]
print(s)
print(sub)
sub = s[::2]
print(sub)
sub = s[::3]
print(sub)
sub = s[::100]
print(sub)
```

```
abcde
ace
ace
ad
a
```

string

- An immutable sequence of Unicode characters.
  - Immutable
  - Sequence
  - Unicode
- Further Reading : Unicode
  - Unicode, formally The Unicode Standard is a text encoding standard maintained by the Unicode Consortium designed to support the use of text in all of the world's writing systems that can be digitized. [1]



1. https://en.wikipedia.org/wiki/Unicode
2. https://home.unicode.org/

string comparison

```python
a = 'a'
b = 'b'
print(f'{a}>{b}={a>b}')

a = 'aa'
b = 'ab'
print(f'{a}>{b}={a>b}')

a = 'aba'
b = 'ab'
print(f'{a}>{b}={a>b}')

a = '0'
b = '9'
print(f'{a}>{b}={a>b}')
```

ASCII code

- **American Standard Code for Information Interchange**
  - 0-127
  - ord()
    - In Python, the ord() function returns the Unicode code point (i.e., an integer) for a given character.
    - It takes a single character as input and provides its corresponding **Unicode value**.
    - The first 128 characters of Unicode are identical to the ASCII character set, meaning Unicode is compatible with ASCII for those first 128 characters.
  - Extended ASCII

```python
print(ord('a'))
print(ord('A'))
print('a'>'b')
print(ord('b'))
print(ord('B'))
print(ord('0'))
print(ord('9'))
print('aba'>'ab')
```

```
97
65
False
98
66
48
57
True
```

ASCII Table

- a→97
- b→98
- A→65
- B→66

Dec	Hx	Oct	Char		Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL	(null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH	(start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX	(start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX	(end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT	(end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ	(enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK	(acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL	(bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS	(backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB	(horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF	(NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT	(vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF	(NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR	(carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO	(shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI	(shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE	(data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1	(device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2	(device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3	(device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4	(device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK	(negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN	(synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB	(end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN	(cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM	(end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB	(substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC	(escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS	(file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS	(group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS	(record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US	(unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

Source: www.LookupTables.com

string built-in methods

Function	Description
find(str, beg=0, end=len(string))	Determine if str occurs in string or in a substring of string if starting index beg and ending index end are given returns index if found and -1 otherwise.
isdigit()	Returns true if string contains only digits and false otherwise.
lower()	Converts all uppercase letters in string to lowercase.
upper()	Converts lowercase letters in string to uppercase.
split(str="", num=string.count(str))	Splits string according to delimiter str (space if not provided) and returns list of substrings; split into at most num substrings if given.
endswith(suffix, beg=0, end=len(string))	Determines if string or a substring of string (if starting index beg and ending index end are given) ends with suffix; returns true if so and false otherwise.
…	

String built-in functions

Name	Description
len(string)	Gives the total length of the string.
max(string)	Returns character from the string with max value.
min(string)	Returns character from the string with min value.

https://www.tutorialspoint.com/python/python_strings.htm

Function and Method

- name = 'Josh'
- Function
  - len(name)
- Method
  - name.lower()
- Further reading
  - Ask ChatGPT
  - Concept of function in python (we will learn this in next lecture)
  - Concept of Class/Object in Object-Oriented Programming

Review

- string
  - Concatenation and repetition
  - String as a sequence
    - Membership operator
    - Slicing
  - Comparison
    - ASCII code
  - Bulit-in functions and methods

List

- A Python list is a sequence of comma separated items, enclosed in square brackets [ ].
  - Name1 = 'Saul'
  - Name2 = 'David'
  - Name3 = 'Solomon'
  - Name4 = 'Rehoboam'
  - Names = ['Saul', 'David', 'Solomon', 'Rehoboam']
- The items in a Python list need not be of the same data type.

```
my_list = ['abc',123,True,[0,1,2]]
print(my_list)
```

```
['abc', 123, True, [0, 1, 2]]
```

List – Concatenation and Repetition

- Concatenation +
  - [1, 2, 3]+[4, 5, 6]
- Repetition *
  - [1, 2, 3]*4
  - ['a', 'b', 'c']*3

```
print([1,2,3]+[4,5,6])
print([1,2,3]*3)
print(['a','b','c']*3)
```

```
[1, 2, 3, 4, 5, 6]
[1, 2, 3, 1, 2, 3, 1, 2, 3]
['a', 'b', 'c', 'a', 'b', 'c', 'a', 'b', 'c']
```

List - Index in a sequence

- my_list[0]
- my_list[-1]
- Membership operator
  - 'abc' in my_list
- Slicing
  - my_list[start : end : step]

```
my_list = ['abc',123,True,[0,1,2],'fff',-1,-100]
print(my_list)
print(my_list[0])
print(my_list[:5:2])
print(my_list[-1])
```

```
['abc', 123, True, [0, 1, 2], 'fff', -1, -100]
abc
['abc', True, 'fff']
-100
```

List

- Update:
  - my_list[1] = 'efg'
  - Can string do this?
    - Cannot
    - String is immutable
- Add
  - my_list.append('hij')
  - my_list.insert(0,'klmn')
- Delete
  - del my_list[-2]
  - my_list.remove('abc')
- Length
  - len(my_list)

```python
my_list = ['abc',123,True,[0,1,2],'fff',-1,-100]
print(my_list)
my_list[1] = 'efg'
print(my_list)
my_list.append('hij')
print(my_list)
my_list.insert(0,'klmn')
print(my_list)
del my_list[-2]
print(my_list)
my_list.remove(True)
print(my_list)
print(len(my_list))
```

```
['abc', 123, True, [0, 1, 2], 'fff', -1, -100]
['abc', 'efg', True, [0, 1, 2], 'fff', -1, -100]
['abc', 'efg', True, [0, 1, 2], 'fff', -1, -100, 'hij']
['klmn', 'abc', 'efg', True, [0, 1, 2], 'fff', -1, -100, 'hij']
['klmn', 'abc', 'efg', True, [0, 1, 2], 'fff', -1, 'hij']
['klmn', 'abc', 'efg', [0, 1, 2], 'fff', -1, 'hij']
7
```

Dead Loop

```python
ll = [0,1,2,3]
for a in ll:
  ll.append(a+1)
  print(ll)
```

```
[0, 1, 2, 3, 1]
[0, 1, 2, 3, 1, 2]
[0, 1, 2, 3, 1, 2, 3]
[0, 1, 2, 3, 1, 2, 3, 4]
[0, 1, 2, 3, 1, 2, 3, 4, 2]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4, 5]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4, 5, 6]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4, 5, 6, 4]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4, 5, 6, 4, 5]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4, 5, 6, 4, 5, 6]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4, 5, 6, 4, 5, 6, 7]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4, 5, 6, 4, 5, 6, 7, 5]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4, 5, 6, 4, 5, 6, 7, 5, 6]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4, 5, 6, 4, 5, 6, 7, 5, 6, 7]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4, 5, 6, 4, 5, 6, 7, 5, 6, 7, 8]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4, 5, 6, 4, 5, 6, 7, 5, 6, 7, 8, 6]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4, 5, 6, 4, 5, 6, 7, 5, 6, 7, 8, 6, 7]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4, 5, 6, 4, 5, 6, 7, 5, 6, 7, 8, 6, 7, 8]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4, 5, 6, 4, 5, 6, 7, 5, 6, 7, 8, 6, 7, 8, 9]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4, 5, 6, 4, 5, 6, 7, 5, 6, 7, 8, 6, 7, 8, 9, 7]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4, 5, 6, 4, 5, 6, 7, 5, 6, 7, 8, 6, 7, 8, 9, 7, 8]
[0, 1, 2, 3, 1, 2, 3, 4, 2, 3, 4, 5, 3, 4, 5, 6, 4, 5, 6, 7, 5, 6, 7, 8, 6, 7, 8, 9, 7, 8, 9]
```

List

- Built-in functions

Name	Description
len(list)	Gives the total length of the list.
max(list)	Returns item from the list with max value.
min(list)	Returns item from the list with min value.

https://www.tutorialspoint.com/python/python_lists.htm

List

- Be careful when you copy a list

```
list1 = [1,2,3,4]
print(list1)
list2 = list1
list2[0] = 100
print(list1)
print(list2)
print(id(list1))
print(id(list2))
```

```
[1, 2, 3, 4]
[100, 2, 3, 4]
[100, 2, 3, 4]
134127354025216
134127354025216
```

Deep Copy

- A deep copy creates a new object and recursively adds copies of nested objects found in the original.
- Changes to the copied object do not affect the original object.

```python
import copy
list1 = [1,2,3,4]
print(list1)
list2 = copy.deepcopy(list1)
list2[0] = 100
print('='*20)
print('after deep copy')
print(f'list1: {list1}, id of list1: {id(list1)}')
print(f'list2: {list2}, id of list2: {id(list2)}')
```

```
[1, 2, 3, 4]
====================
after deep copy
list1: [1, 2, 3, 4], id of list1: 133044374098624
list2: [100, 2, 3, 4], id of list2: 133044374126336
```

Shallow Copy

- A shallow copy creates a new object but inserts references into it to the objects found in the original.
- Changes to the copied object can affect the original object if the copied object contains references to mutable objects.

  - A reference is a variable that points to or "references" a location in memory where an object is stored.

Shallow Copy

```python
import copy
list1 = [[1],[2],3,4]
list3 = copy.copy(list1)
print(id(list1))
print(id(list3))
print('id of items 0')
print(id(list1[0]))
print(id(list3[0]))
print('id of items 1')
print(id(list1[1]))
print(id(list3[1]))
```

```
140095038282880
140095023949568
id of items 0
140095024137792
140095024137792
id of items 1
140095038866560
140095038866560
```

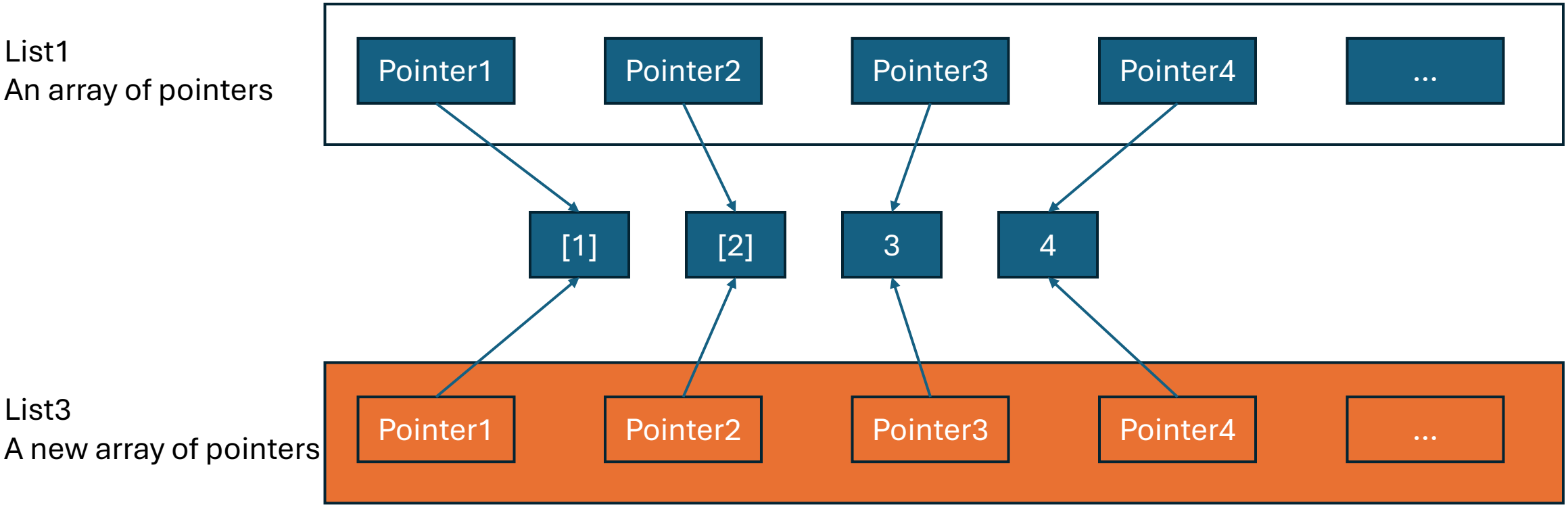
Changes to the copied object can affect the original object if the copied object contains references to mutable objects.

```python
import copy
list1 = [[1],[2],3,4]
list3 = copy.copy(list1)
list3[0][0] = 100
print('new values')
print(list1)
print(list3)
print('id of the first item')
print(id(list1[0]))
print(id(list3[0]))
print('\n')
#============================
list3[0] = [200]
print('new values')
print(list1)
print(list3)
print('id of the first item')
print(id(list1[0]))
print(id(list3[0]))
```

```
new values
[[100], [2], 3, 4]
[[100], [2], 3, 4]
id of the first item
133044373899072
133044373899072


new values
[[100], [2], 3, 4]
[[200], [2], 3, 4]
id of the first item
133044373899072
133044373903616
```

Shallow Copy

A shallow copy creates a new object but inserts references into it to the objects found in the original.



Immutable vs Mutable

- Immutable: The value of the object cannot be changed once it is created, such as strings, tuples, and integers.
- Mutable: The value of the object can be changed, such as in lists, dictionaries, and sets.
  - **You can change the individual items, not the entire object at once**

```
[2] name = 'john'
    print(f'{name=}, {id(name)=}')
    name = 'jane'
    print(f'{name=}, {id(name)=}')
```

```
name='john', id(name)=134128136879536
name='jane', id(name)=134128136879600
```

```
a = [1,2,3,4]
print(id(a[0]))
print(id(a))
print('='*20)

a[0] = 100
print(id(a[0]))
print(id(a))
print('='*20)

a = [1,1,1]
print(id(a))
```

```
132413518217456
132413306762880
====================
132413518220624
132413306762880
====================
132413088143680
```

1. https://www.toppr.com/guides/python-guide/references/methods-and-functions/methods/built-in/python-id/

Review

- Operations on a sequence
  - Concatenation and repetition
  - append, del, len, etc.
- Assignment, Deep Copy and Shallow Copy
  - **Sometimes you may change the values of a list unexpectedly**
    - copy.deepcopy
  - Immutable and Mutable
    - **You can change the individual items, not the entire object at once**

Tuples

- A sequence of immutable Python objects
- Difference between tuple and list
  - Tuples use parentheses () and lists use square brackets []
  - The tuples can not be changed

```
tp1 = (1,[2,4,5],6)
print(tp1)
tp1[1][0] = 3
print(tp1)
tp1[1] = [3,4,5]
print(tp1)
```

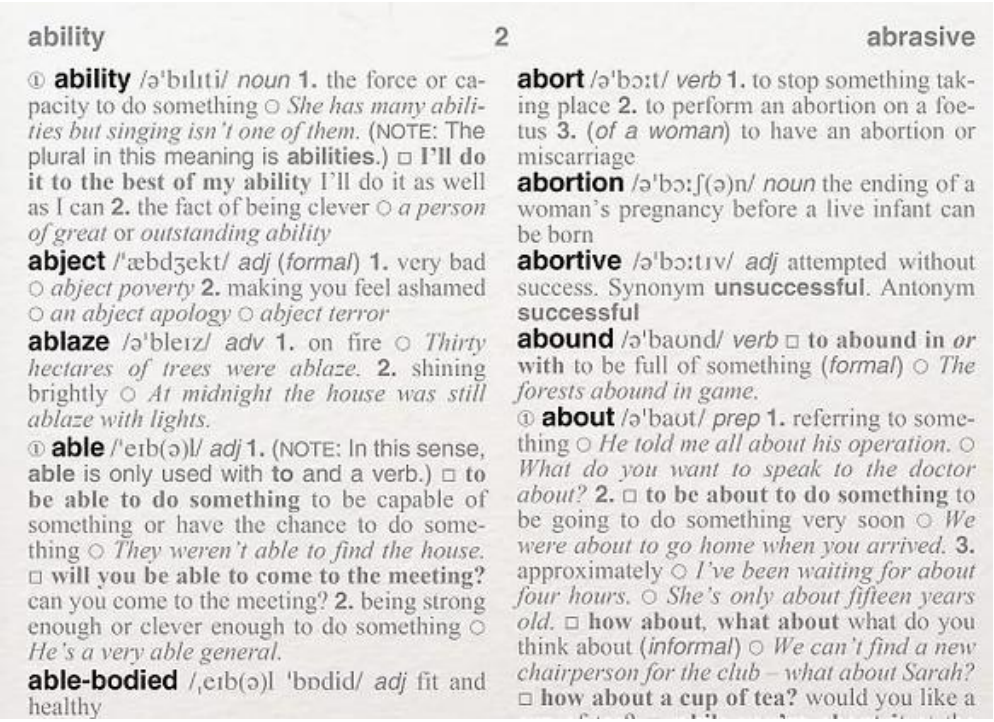
```
(1, [2, 4, 5], 6)
(1, [3, 4, 5], 6)
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-65-5ef71e1020a9> in <cell line: 5>()
      3 tp1[1][0] = 3
      4 print(tp1)
----> 5 tp1[1] = [3,4,5]
      6 print(tp1)

TypeError: 'tuple' object does not support item assignment
```

Next steps: Explain error

Dictionary

- A word and its explanation
- In Python, a dictionary is a built-in data type that stores data in key-value pairs.[2]
- Each key in a dictionary is unique and maps to a value.
  - Example:

ID	Name	Score
001	John	100+
002	John	80
…		



ability 2 abrasive

① **ability** /ə'bɪlɪti/ *noun* **1.** the force or capacity to do something ○ *She has many abilities but singing isn't one of them.* (NOTE: The plural in this meaning is **abilities**.) □ **I'll do it to the best of my ability** I'll do it as well as I can **2.** the fact of being clever ○ *a person of great* or *outstanding ability*

abject /'æbdʒekt/ *adj* (*formal*) **1.** very bad ○ *abject poverty* **2.** making you feel ashamed ○ *an abject apology* ○ *abject terror*

ablaze /ə'bleɪz/ *adv* **1.** on fire ○ *Thirty hectares of trees were ablaze.* **2.** shining brightly ○ *At midnight the house was still ablaze with lights.*

① **able** /'eɪb(ə)l/ *adj* **1.** (NOTE: In this sense, **able** is only used with **to** and a verb.) □ **to be able to do something** to be capable of something or have the chance to do something ○ *They weren't able to find the house.* □ **will you be able to come to the meeting?** can you come to the meeting? **2.** being strong enough or clever enough to do something ○ *He's a very able general.*

able-bodied /ˌeɪb(ə)l 'bɒdɪd/ *adj* fit and healthy

abort /ə'bɔːt/ *verb* **1.** to stop something taking place **2.** to perform an abortion on a foetus **3.** (*of a woman*) to have an abortion or miscarriage

abortion /ə'bɔːʃ(ə)n/ *noun* the ending of a woman's pregnancy before a live infant can be born

abortive /ə'bɔːtɪv/ *adj* attempted without success. Synonym **unsuccessful**. Antonym **successful**

abound /ə'baʊnd/ *verb* □ **to abound in** *or* **with** to be full of something (*formal*) ○ *The forests abound in game.*

① **about** /ə'baʊt/ *prep* **1.** referring to something ○ *He told me all about his operation.* ○ *What do you want to speak to the doctor about?* **2.** □ **to be about to do something** to be going to do something very soon ○ *We were about to go home when you arrived.* **3.** approximately ○ *I've been waiting for about four hours.* ○ *She's only about fifteen years old.* □ **how about, what about** what do you think about (*informal*) ○ *We can't find a new chairperson for the club – what about Sarah?* □ **how about a cup of tea?** would you like a

1. https://www.wikihow.com/Use-a-Dictionary
2. https://tutorialspoint.com/python/python_dictionary.htm

Dictionary

- Each key is separated from its value by a colon (:)
  - key : value ,
- The items are separated by commas
- The whole thing is enclosed in curly braces
  - An empty dictionary is {}

```
students = {'000':'John', '001':'Jane', '002':'Josh', '003':'James'}
print(students)
```

```
{'000': 'John', '001': 'Jane', '002': 'Josh', '003': 'James'}
```

Dictionary

- Keys are unique within a dictionary
- Values may not be unique
- Values can be of any type
  - Strings, numbers, tuples, lists, dictionaries etc.
- Keys must be of an **immutable** data type
  - Strings, numbers, or tuples

```
students = {'000':'John', '001':80, '002':['Jane',90], '003':['Jane',90]}
print(students)
print(students['002'])
print(students['002'][1])
```

```
{'000': 'John', '001': 80, '002': ['Jane', 90], '003': ['Jane', 90]}
['Jane', 90]
90
```

Dictionary*

- Keys must be immutable → Why?
- Keys must be unique
  - Hashing ensures quick lookup.
  - Uniqueness ensures no conflicts in meaning (no duplicate keys).
- Hash function
  - A function that takes an input (number, string, object) and returns a fixed-size integer called the hash value.
- Dictionary is implemented as a hash table.
- The key's hash value is used to decide where the value is stored.
  - If the key were mutable, its hash could change → the dictionary would "lose" the item.

Dictionary

- Access
  - value = students[key]
- Update and Add
  - students[key] = value
  - If a key already exists, the old value will be overwritten by the new value.
  - If it is a new key, the key and value pair will be added to the dictionary
- Delete
  - del students[key]
  - students.clear()

```
students = {'000':'John','001':'John','002':'Josh','003':'Jack'}
print(students)
print(students['000'])
print(students['001'])

students['001'] = 'John_no2'
print(students['001'])

print(students)
students['004'] = 'Jason'
print(students)
```

```
{'000': 'John', '001': 'John', '002': 'Josh', '003': 'Jack'}
John
John
John_no2
{'000': 'John', '001': 'John_no2', '002': 'Josh', '003': 'Jack'}
{'000': 'John', '001': 'John_no2', '002': 'Josh', '003': 'Jack', '004': 'Jason'}
```

Built-in Functions with Dictionaries

Name	Description
len(dict)	Give the length of the dictionary, which is the number of items in the dictionary
str(dict)	Produce a printable string representation of a dictionary

Python Dictionary Methods

Name	Description
dictionary.keys()	Returns list of dictionary keys
dictionary.values()	Returns list of dictionary values
dictionary.items()	Returns list of dictionary items: (key, value) tuple pairs
dictionary.update(dictionary2)	Adds dictionary2's key-values pairs to the first dictionary

```python
students = {'000':'John', '001':80, '002':['Jane',90], '003':['Jane',90]}
print(students.keys())
print(list(students.keys())[0])
print(students.values())
print(students.items())
teachers = {'000':'Tom','004':'Zhang'}
students.update(teachers)
print(students)
```

```
dict_keys(['000', '001', '002', '003'])
000
dict_values(['John', 80, ['Jane', 90], ['Jane', 90]])
dict_items([('000', 'John'), ('001', 80), ('002', ['Jane', 90]), ('003', ['Jane', 90])])
{'000': 'Tom', '001': 80, '002': ['Jane', 90], '003': ['Jane', 90], '004': 'Zhang'}
```

Iterate through all the elements

- Practice
  - Create a dictionary, which contains student scores
  - Key is the student number
  - Value is the score
  - Add 10 points to each student's score
    - For loop
    - dictionary.keys()
  - ==========
  - Sort the scores in descending order
  - Print out the names and scores
  - Sorted function
    - You can sort based on different conditions by providing a **custom key function**
    - For example, you can sort by length, or even based on specific attributes of objects

Review

- Key-value pairs
- Keys are unique
- Add, update, delete
- for key, value in my_dict.items():
- keys(), values()

List Comprehension

- **Generate a new list** by applying an expression to each item in an **existing iterable** (like a list or range) in **a single line of code**.
  - Create a list containing the first ten perfect squares

```
squares = []
for i in range(10):
  squares.append(i**2)
print(squares)
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```
squares = [i**2 for i in range(10)]
print(squares)
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

List Comprehension

- new_list = [expression for member in iterable]
  - expression: the list member itself or any valid expression returns a value
    - i**2
  - member: the object or value in the list or iterable
    - i
  - iterable: a list, set, sequence, generator or any other object that can return its elements one at a time
    - list(range(10)) : [0,1,2,3,4,5,6,7,8,9]

```
squares = [i**2 for i in range(10)]
print(squares)
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

List Comprehension

- Using conditional logic
- new_list= [expression for member in iterable (if conditional)]

```
squares = [i**2 for i in range(10) if i<5 ]
print(squares)
```

```
[0, 1, 4, 9, 16]
```

```
squares = [i**2 if i<5 else i for i in range(10) ]
print(squares)
```

```
[0, 1, 4, 9, 16, 5, 6, 7, 8, 9]
```

```
squares = [i**2 if i<5 else (i+100 if i<8 else i) for i in range(10) ]
print(squares)
```

```
[0, 1, 4, 9, 16, 105, 106, 107, 8, 9]
```

Readability

Integrated application

```
students = {'000':['John',80], '001':['Jane',90], '002':['James',85], '003':['Jen',70]}
scores = [info[1] for info in students.values()]
print(scores)
```

```
[80, 90, 85, 70]
```

Why Use List Comprehension?

- Faster
  - Implemented in C, avoids Python-level overhead.
- Concise
  - One line replaces multiple lines of for + append.
- Readable
  - Directly shows the intent — “generate a list of …”.
- Expressive
  - Supports filtering, conditional expressions, nested loops.
- Consistent
  - Same style works for list, set, dict, generator.

Faster

```python
import time

n = 10_000_000

start = time.time()
out = []
for x in range(n):
    if x % 2 == 0:
        out.append(x * x)
end = time.time()

print("for + append + if:", end - start, "seconds")
```

```
for + append + if: 1.638904094696045 seconds
```

```python
import time

n = 10_000_000

start = time.time()
out = [x * x for x in range(n) if x % 2 == 0]
end = time.time()

print("list comprehension + if:", end - start, "seconds")
```

```
list comprehension + if: 0.9551346302032471 seconds
```

Review: f-string vs .format*

```python
import time

user = "alice"
ip = "192.168.0.1"
timestamp = "2025-09-14 21:00:00"
endpoint = "/api/v1/data"
status = 200

start = time.time()
for _ in range(1_000_000):
    log = f"[{timestamp}] User={user} IP={ip} accessed {endpoint} with status={status}"
end = time.time()

print("f-string time:", end - start)
```

```
f-string time: 0.4094526767730713
```

```python
import time

user = "alice"
ip = "192.168.0.1"
timestamp = "2025-09-14 21:00:00"
endpoint = "/api/v1/data"
status = 200

start = time.time()
for _ in range(1_000_000):
    log = "[{}] User={} IP={} accessed {} with status={}".format(timestamp, user, ip, endpoint, status)
end = time.time()

print(".format time:", end - start)
```

```
.format time: 0.7212386131286621
```

END